

NT219- Cryptography

Week 12: Hash Function and Message Authentication Codes

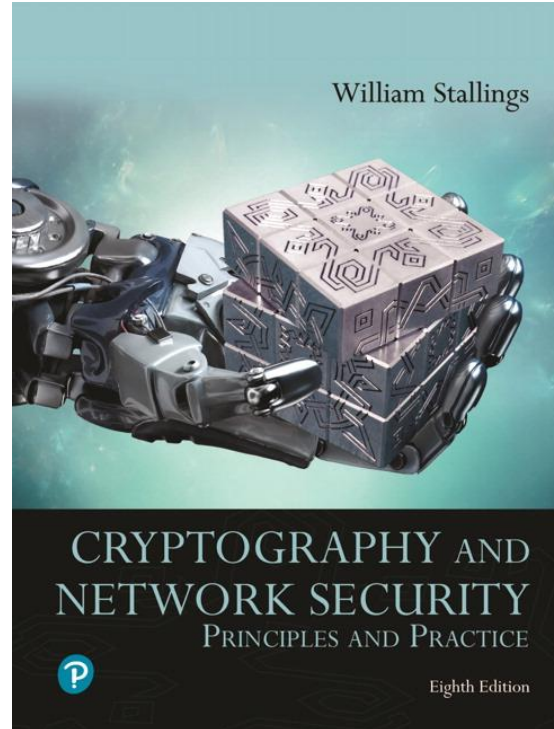
PhD. Ngoc-Tu Nguyen

tunn@uit.edu.vn

- Motivations
- Hash function
 - CRC
- Cryptographic Hash function
 - SHA2
 - **SHA3**
- **Message authentication code**

Textbooks and References

- Text books



[1] Chapter 11,12

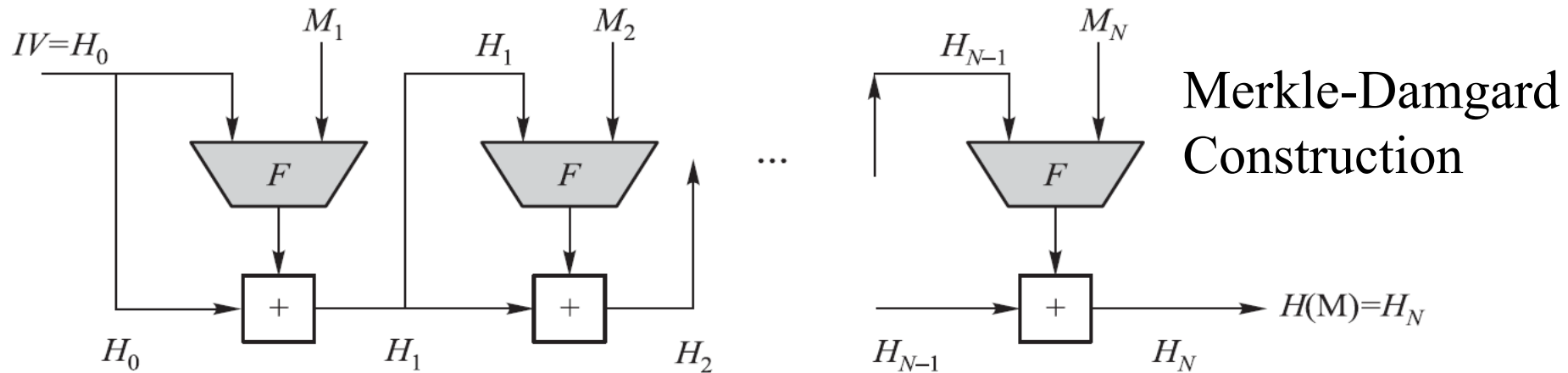
SHA-1, SHA-2, SHA-3

Algorithm and variant		Output size (bits)	Internal state size (bits)	Block size (bits)	Rounds	Operations	Security (in bits) against collision attacks	Capacity against length extension attacks
MD5 (as reference)		128	128 (4 × 32)	512	64	And, Xor, Rot, Add (mod 2 ³²), Or	≤18 (collisions found) ^[2]	0
SHA-0		160	160 (5 × 32)	512	80	And, Xor, Rot, Add (mod 2 ³²), Or	<34 (collisions found)	0
SHA-1							<63 (collisions found) ^[3]	
SHA-2	SHA-224	224	256 (8 × 32)	512	64	And, Xor, Rot, Add (mod 2 ³²), Or, Shr	112	32
	SHA-256	256					128	0
	SHA-384	384	512 (8 × 64)	1024	80	And, Xor, Rot, Add (mod 2 ⁶⁴), Or, Shr	192	128 (≤ 384)
	SHA-512	512					256	0
	SHA-512/224	224					112	288
	SHA-512/256	256					128	256
SHA-3	SHA3-224	224	1600 (5 × 5 × 64)	1152	24 ^[4]	And, Xor, Rot, Not	112	448
	SHA3-256	256		1088			128	512
	SHA3-384	384		832			192	768
	SHA3-512	512		576			256	1024
	SHAKE128	d (arbitrary)		1344			min(d/2, 128)	256
	SHAKE256	d (arbitrary)		1088			min(d/2, 256)	512

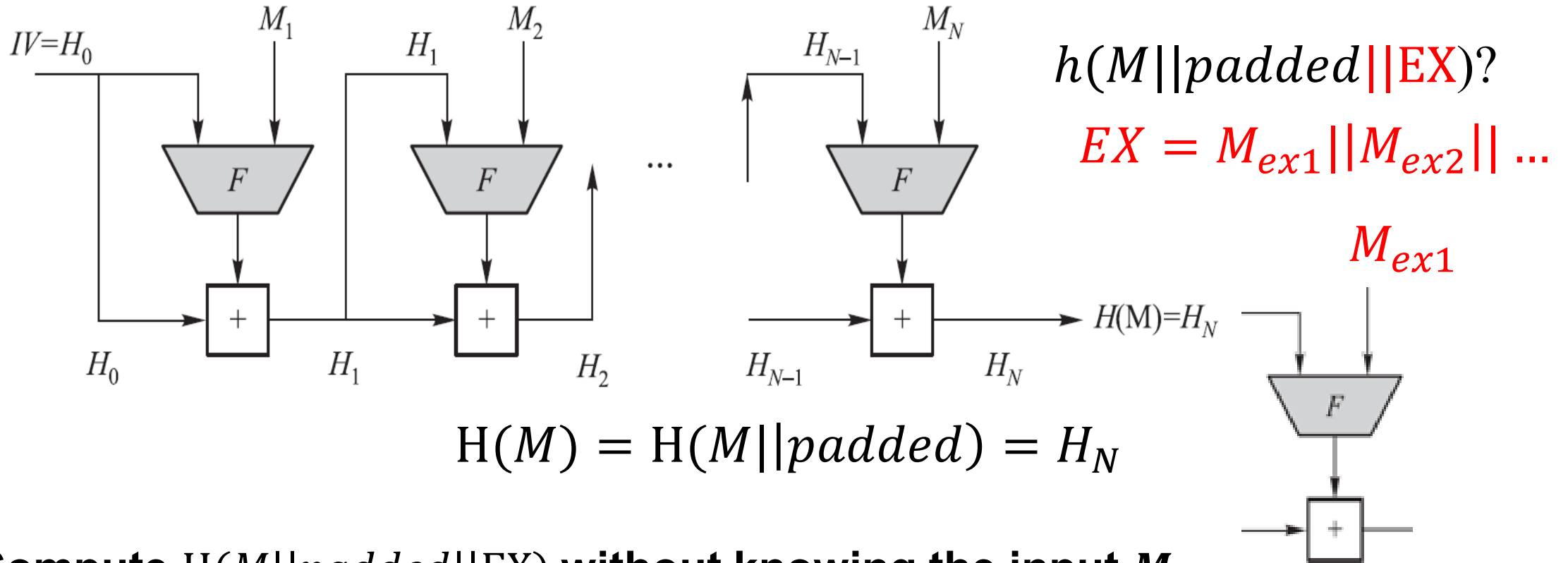
https://en.wikipedia.org/wiki/Secure_Hash_Algorithm

SHA-1, SHA-2

Algorithm	Message Size (bits)	Block Size (bits)	Word Size (bits)	Message Digest Size (bits)
SHA-1	$< 2^{64}$	512	32	160
SHA-224	$< 2^{64}$	512	32	224
SHA-256	$< 2^{64}$	512	32	256
SHA-384	$< 2^{128}$	1024	64	384
SHA-512	$< 2^{128}$	1024	64	512
SHA-512/224	$< 2^{128}$	1024	64	224
SHA-512/256	$< 2^{128}$	1024	64	256



Length extension attack on SHA2

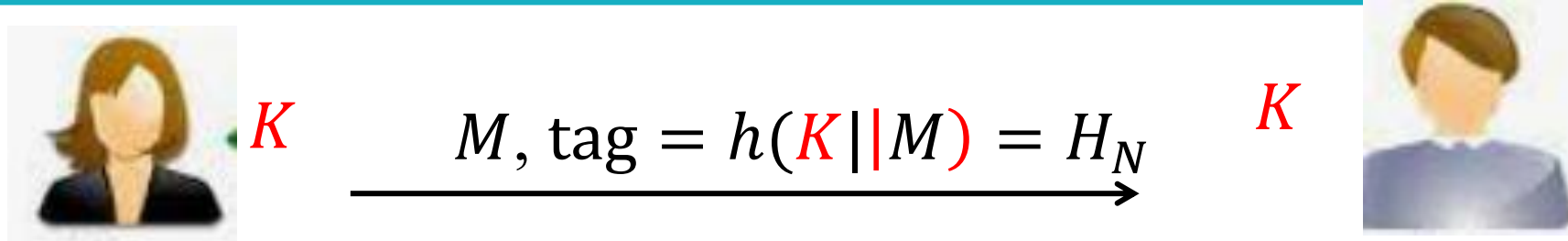


➤ **Compute $H(M || padded || EX)$ without knowing the input M**

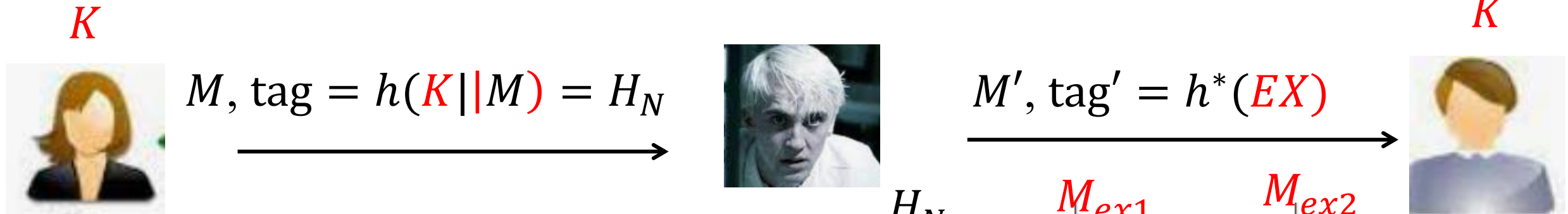
$$H(M || padded || EX) = H^*(EX),$$

where IV of $H^* = H_N$

Length extension attack on SHA2

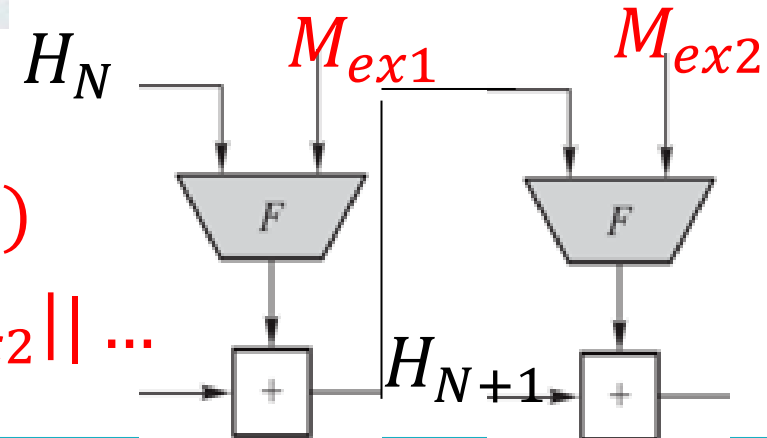


$$M' = M || \text{padding} || EX, \text{tag}' = h(K || M') = h^*(EX)$$



$$h(K || M') = h(K || M || \text{padding} || EX) = h^*(EX)$$

$$EX = M_{ex1} || M_{ex2} || \dots$$



SHA3 Standard

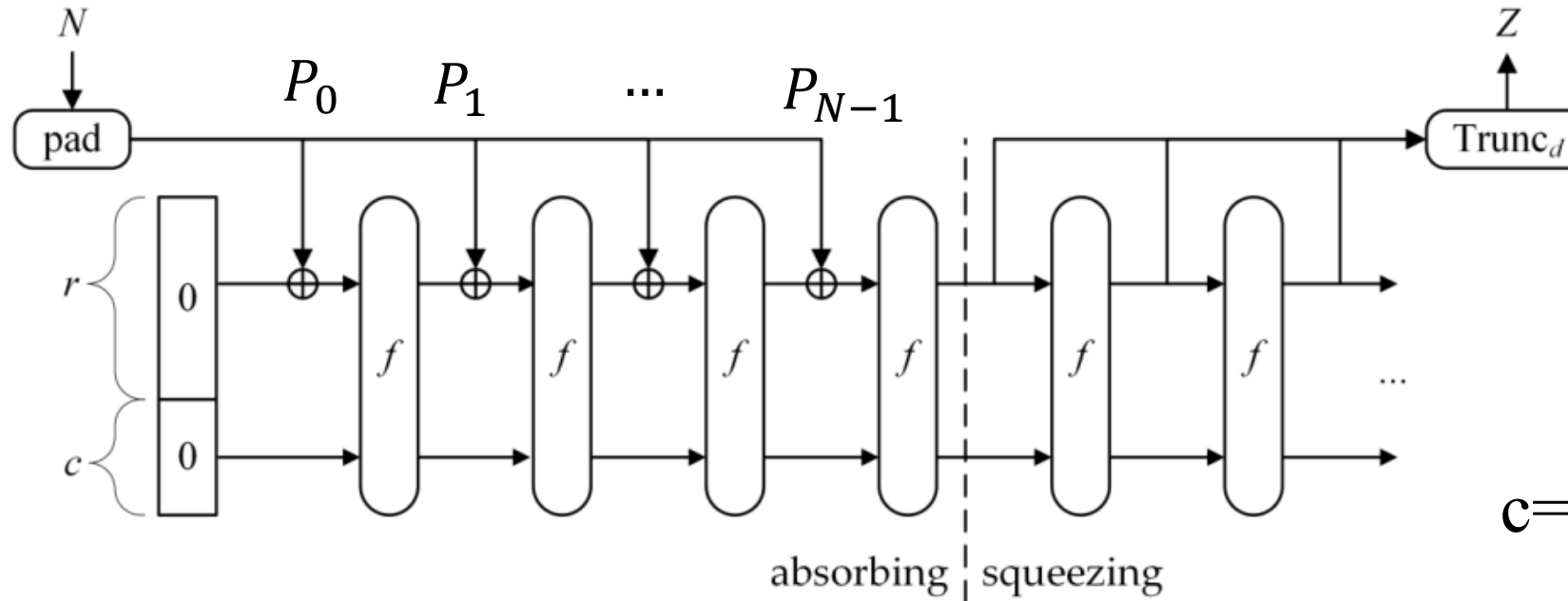
- SHA-3 provides an alternative to SHA-2, and is drop-in compatible with any system using SHA-2
- SHA-3 uses a **sponge construction**, instead of the CBC mode of repeated compressions used by SHA-1, SHA-2, and Whirlpool

<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf>

- 2007: Request for submissions of new hash functions
- 2008: Submissions deadline. Received 64 entries. Announced first-round selections of 51 candidates.
- 2009: After First SHA-3 candidate conference in Feb, announced 14 Second Round Candidates in July.
- 2010: After one year public review of the algorithms, hold second SHA-3 candidate conference in Aug. Announced 5 Third-round candidates in Dec.
- 2011: Public comment for final round
- 2012: October 2, NIST selected SHA3
 - Keccak (pronounced “catch-ack”) created by Guido Bertoni, Joan Daemen and Gilles Van Assche, Michaël Peeters

SHA3 Standard

Sponge construction (Keccak)

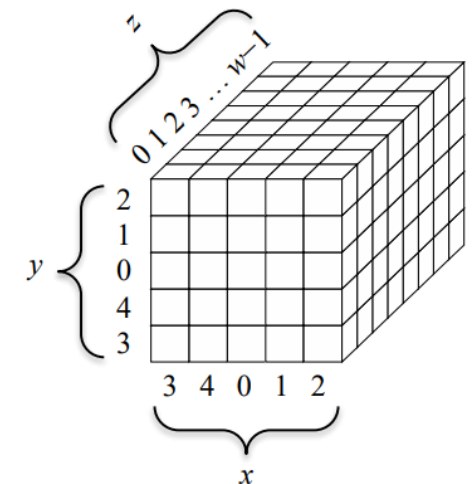


r (block size)



$c=2d = 2 \cdot (\text{output length})$

- Let M be the input string; d = the hash length.
- $b = r + c$, where $c = 2d$
 - ✓ r is called **rate** and c **capacity**
- Where $b = 25 \times 2^l$ with $0 \leq l \leq 6$
 $b \in \{25, 50, 100, 200, 400, 800, 1600\}$



Bertoni, G., Daemen, J., Peeters, M., Van Assche, G., & Van Keer, R. (2012). Keccak implementation overview. URL: <http://keccak.neokeon.org/Keccak-implementation-3.2.pdf>.

Example

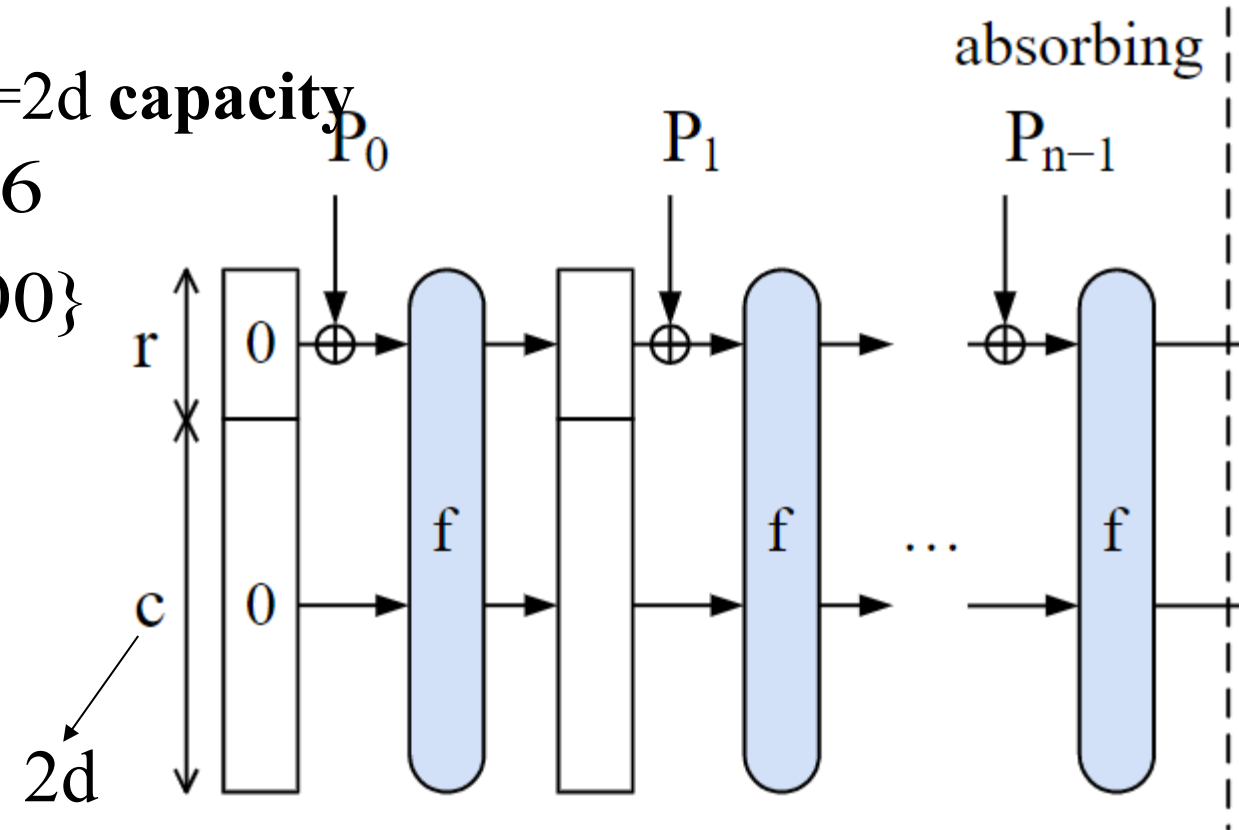
- Let M be the input string; d = the hash length.
- $b = r + c$, where $c = 2d$
 - ✓ r (**block size**) is called **rate**, and $c=2d$ **capacity**
- Where $b = 25 \times 2^l$ with $0 \leq l \leq 6$

$b \in \{25, 50, 100, 200, 400, 800, 1600\}$

Ex. $d = 512$, then $c = 1024$.

Choose $b = 1600$, then $r = 576$.

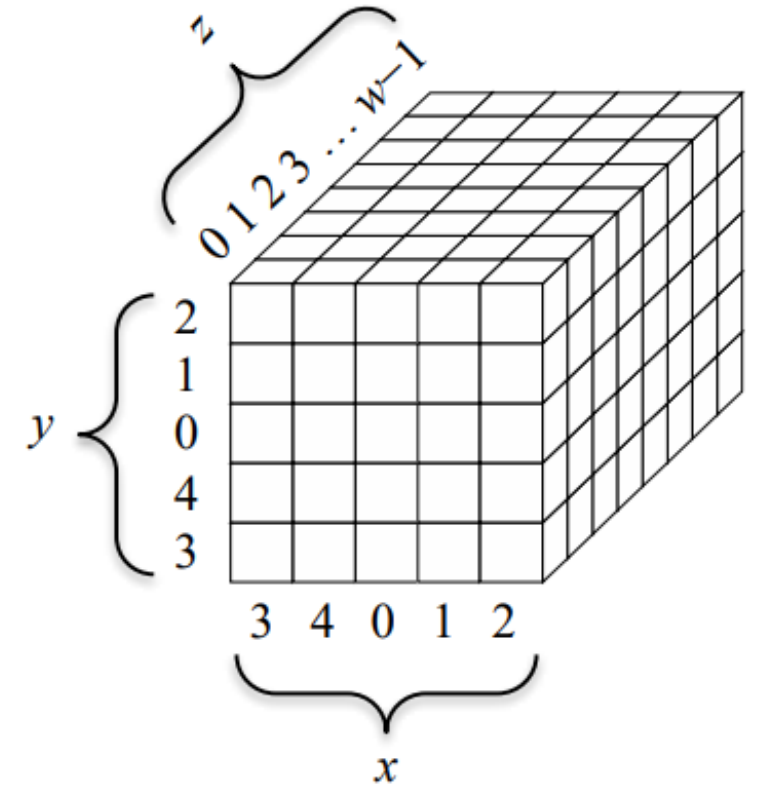
Instance	Output size d	Rate r = block size
SHA3-256(M)	256	1088
SHA3-384(M)	384	832
SHA3-512(M)	512	576



$$\| P_0 \| = \| P_1 \| = \dots \| P_{N-1} \| = r$$

Setup

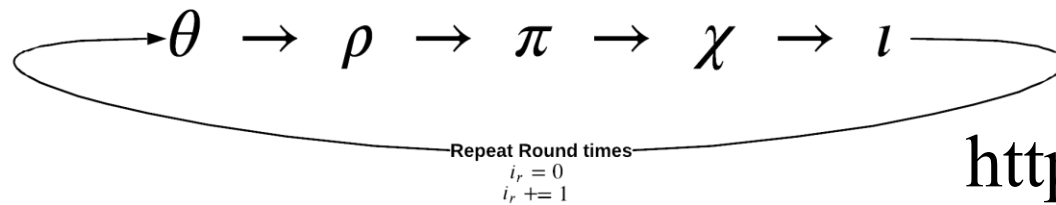
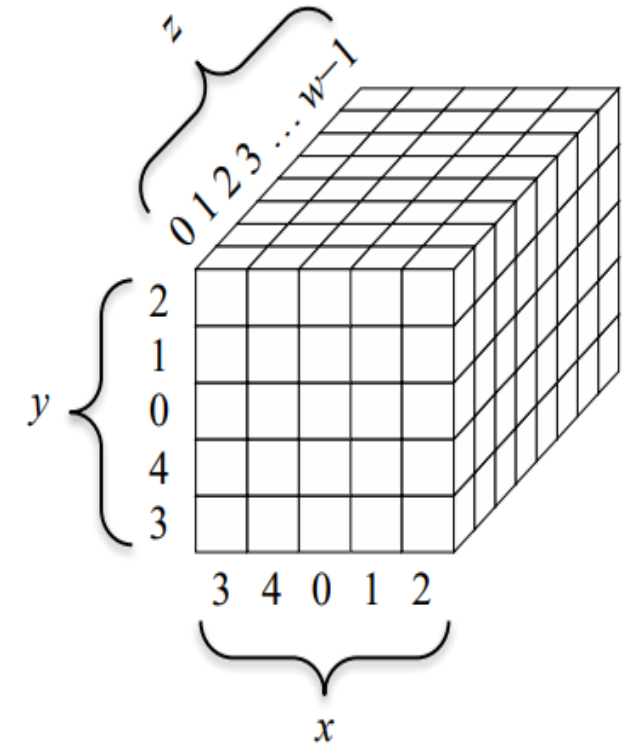
- Pad M by appending $1\{0\}^*1$ to produce M' such that $|M'|$ is divisible by r .
- Divide M' into $N = |M'|/r$ blocks: $M_1, \dots, M_N$
- Let A be a b -bit string and denote A as a 5×5 matrix
- Let $a_{i,j,k}$ denote the k^{th} bit in $a_{i,j}$
- Let f_b be a fixed-length permutation on b -bit inputs
- Let $p_r = \text{pfx}_r, s_c = \text{sfx}_c$



$$A = (a_{i,j,k})_{5 \times 5 \times w}$$

Absorb and Squeeze

Function	Type	Description
θ	Substitution	New value of each bit in each word depends on its current value and on one bit in each word of preceding column and one bit of each word in succeeding column.
ρ	Permutation	The bits of each word are permuted using a circular bit shift. $W[0, 0]$ is not affected.
π	Permutation	Words are permuted in the 5×5 matrix. $W[0, 0]$ is not affected.
χ	Substitution	New value of each bit in each word depends on its current value and on one bit in next word in the same row and one bit in the second next word in the same row.
ι	Substitution	$W[0, 0]$ is updated by XOR with a round constant.



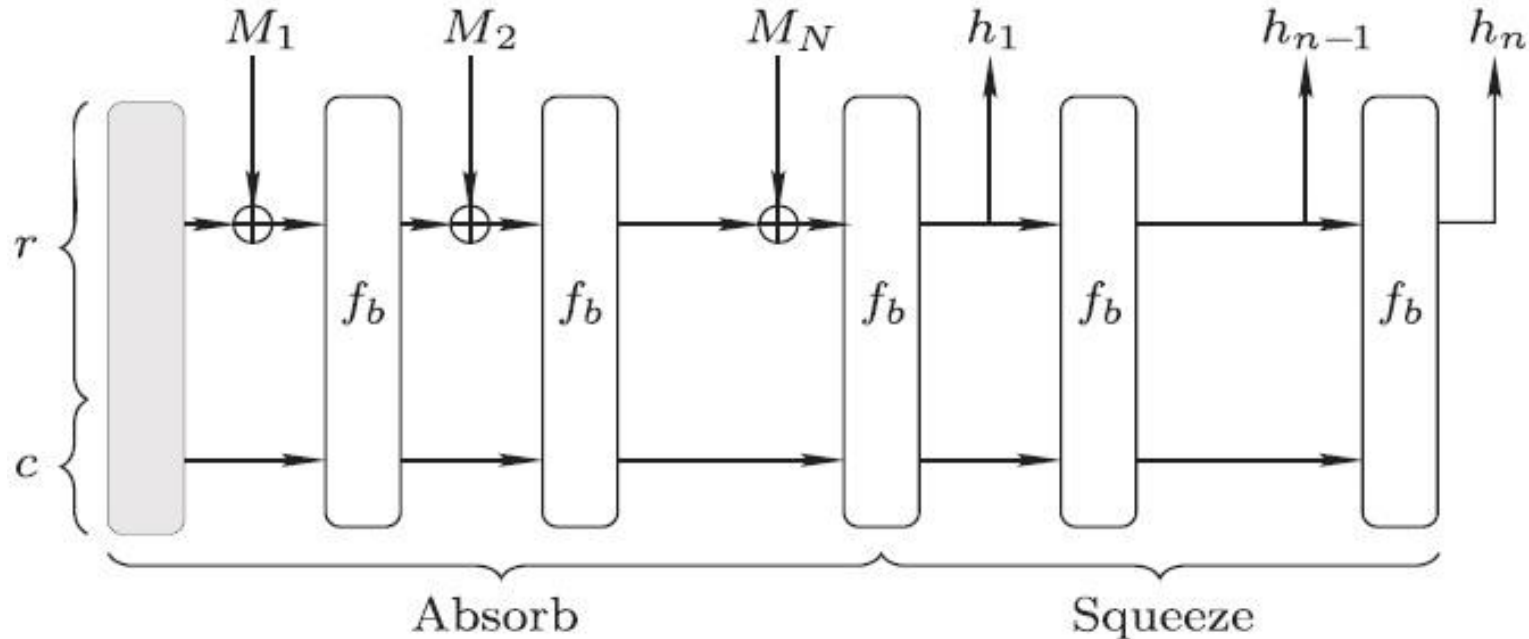
<https://chemejon.io/sha-3-explained/>

$$\text{Rounds} = 12 + 2\ell$$

Absorb and Squeeze

Absorb: $A_i = f_b((p_r(M_i \oplus A_{i-1}) \parallel s_c(A_{i-1})),$
 $i = 1, \dots, N.$

Squeeze: $A_{N+i} = f_b(A_{N+i-1}), \quad h_i = p_r(A_{N+i-1}),$
 $i = 1, \dots, n. \quad i = 1, \dots, n.$



Permutation $f_b(b)$

The KECCAK Family of Permutations

We now describe the KECCAK family of permutations f_b , where $b = 25 \times 2^\ell$ with $0 \leq \ell \leq 6$, and $b = r + c$.

The permutation f_b takes a 5×5 state matrix $\mathbf{A}$ as input and carries out the following five operations, where indices are computed modulo 4 and 2^ℓ where appropriate.

1. *Diffusion*: For all $0 \leq i, j \leq 4$ and $0 \leq k \leq 2^\ell - 1$, compute

$$a_{i,j,k} = a_{i,j,k} \oplus \bigoplus_{y=0}^4 a_{i-1,y,k} \oplus \bigoplus_{y=0}^4 a_{i+1,y,k-1}.$$

SHA-3 Hash

2. *Dispersion*(of bits in words): This operation is the following sequence of 24 steps:

- a) Set $i = 1$ and $j = 0$.
- b) For $t = 0$ to 23 do
 - i. $a_{i,j} = a_{i,j} \oplus ((t + 1)(t + 2) / 2)$.
 - ii. Set $i = j$ and $j = (2i + 3j) \bmod 5$.

3. *Dispersion*(of words): For all $0 \leq i, j \leq 4$, compute

$$a_{i,(2i+3j) \bmod 5} = a_{i,j}.$$

4. *Nonlinear Map*: For each $0 \leq i, j \leq 4$, compute

$$a_{i,j} = a_{i,j} \oplus (\overline{a_{(i+1) \bmod 5,j}} \wedge a_{(i+2) \bmod 5,j}).$$

This map provides resistance to linear cryptanalysis.

5. *Symmetry Disruption*: During the l -th round, compute

$$a_{0,0} = a_{0,0} \oplus \text{RC}_l,$$

where RC_l is the round constant for the l -th round.

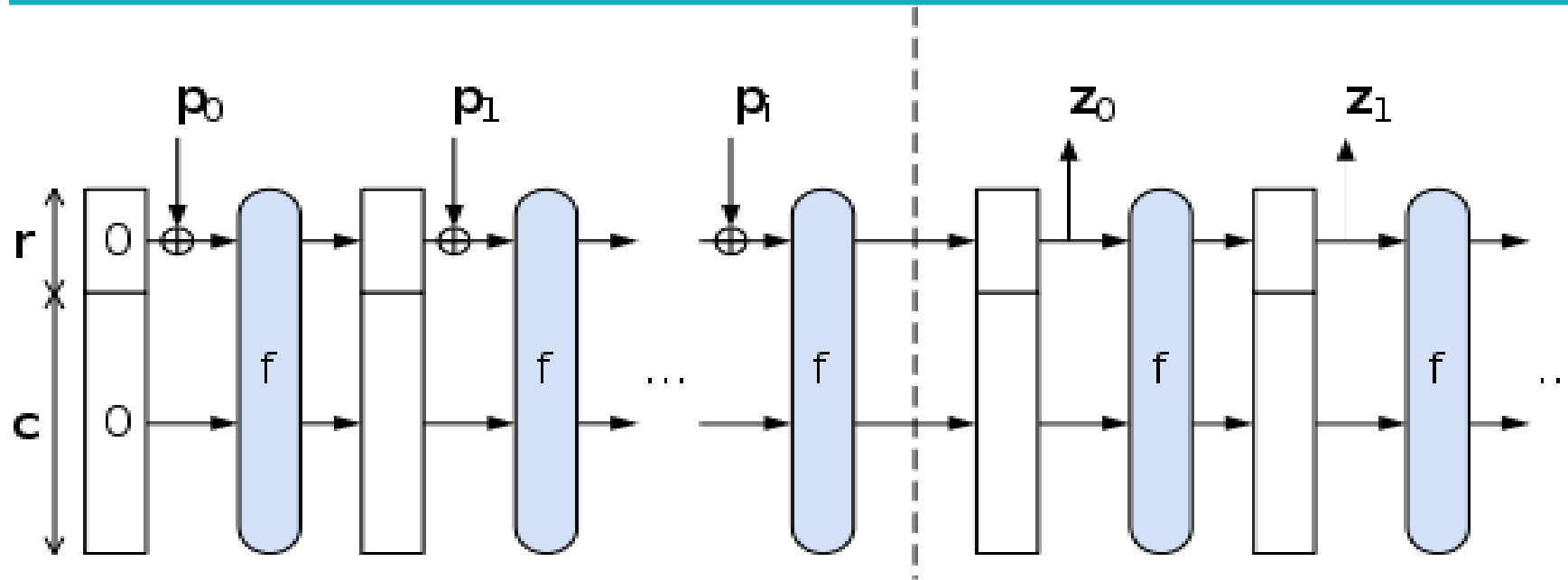
Table 4.2 lists the round constants for $\ell = 6$, where the length of each word is 64-bit long.

SHA-3 Hash

Table 4.2: Round constants for the symmetry disruption phase of the f_b for $\ell = 6$. For $\ell < 6$, use the prefix of these round constants to obtain the round constants of appropriate length

l	Value	l	Value
0	0x0000000000000001	12	0x000000008000808B
1	0x0000000000008082	13	0x800000000000008B
2	0x800000000000808A	14	0x8000000000008089
3	0x8000000080008000	15	0x8000000000008003
4	0x000000000000808B	16	0x8000000000008002
5	0x0000000080000001	17	0x8000000000000080
6	0x8000000080008081	18	0x000000000000800A
7	0x8000000000008009	19	0x800000008000000A
8	0x000000000000008A	20	0x8000000080008081
9	0x0000000000000088	21	0x8000000000008080
10	0x0000000080008009	22	0x0000000080000001
11	0x000000008000000A	23	0x8000000080008008

The Sponge Construction: Used by SHA-3



- Each round, the next r bits of message is XOR'ed into the first r bits of the state, and a function f is applied to the state.
- After message is consumed, output r bits of each round as the hash output; continue applying f to get new states
- SHA-3 uses 1600 bits for state size

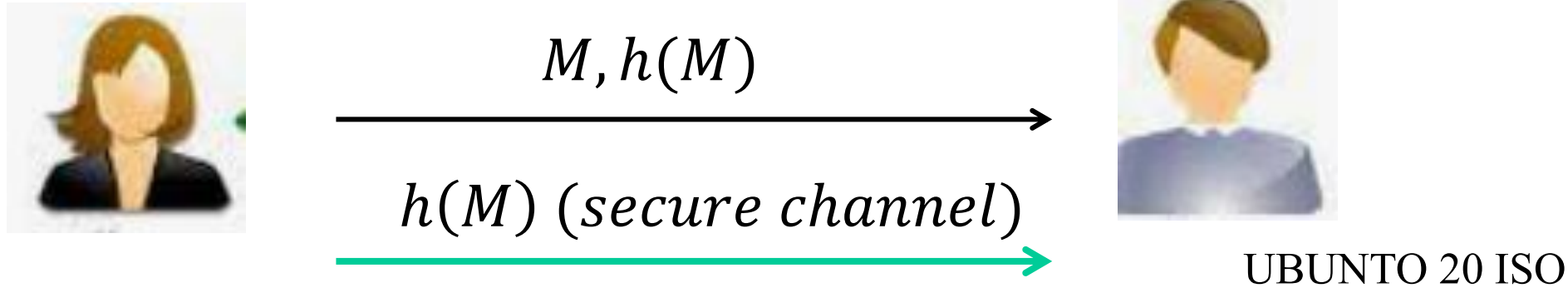
Choosing the length of Hash outputs

- The Weakest Link Principle:
 - A system is only as secure as its weakest link.
- Hence all links in a system should have similar levels of security.
- Because of the birthday attack, the length of hash outputs in general should double the key length of block ciphers
 - SHA-224 matches the 112-bit strength of triple-DES (encryption 3 times using DES)
 - SHA-256, SHA-384, SHA-512 match the new key lengths (128,192,256) in AES
 - SHAKE: output length d

Limitation of Using Hash Functions for Authentication

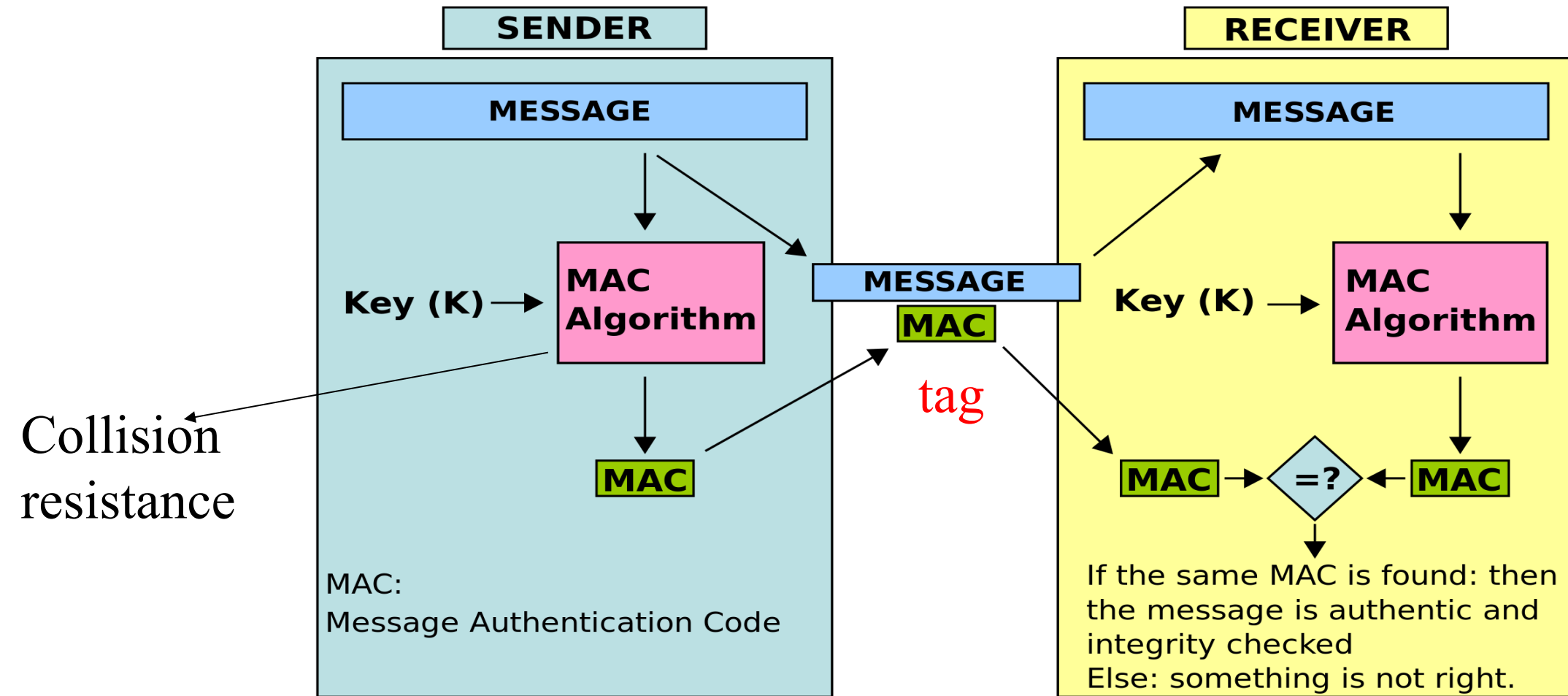
- Is this secure scheme (M cannot be modified)?

- Case 1:



- Require an authentic channel to transmit the hash of a message
 - Without such a channel, it is insecure, because anyone can compute the hash value of any message, as the hash function is public
 - Such a channel may not always exist
- How to address this?
 - use more than one hash functions
 - use a key to select which one to use

Message Authentication Code



https://en.wikipedia.org/wiki/Message_authentication_code

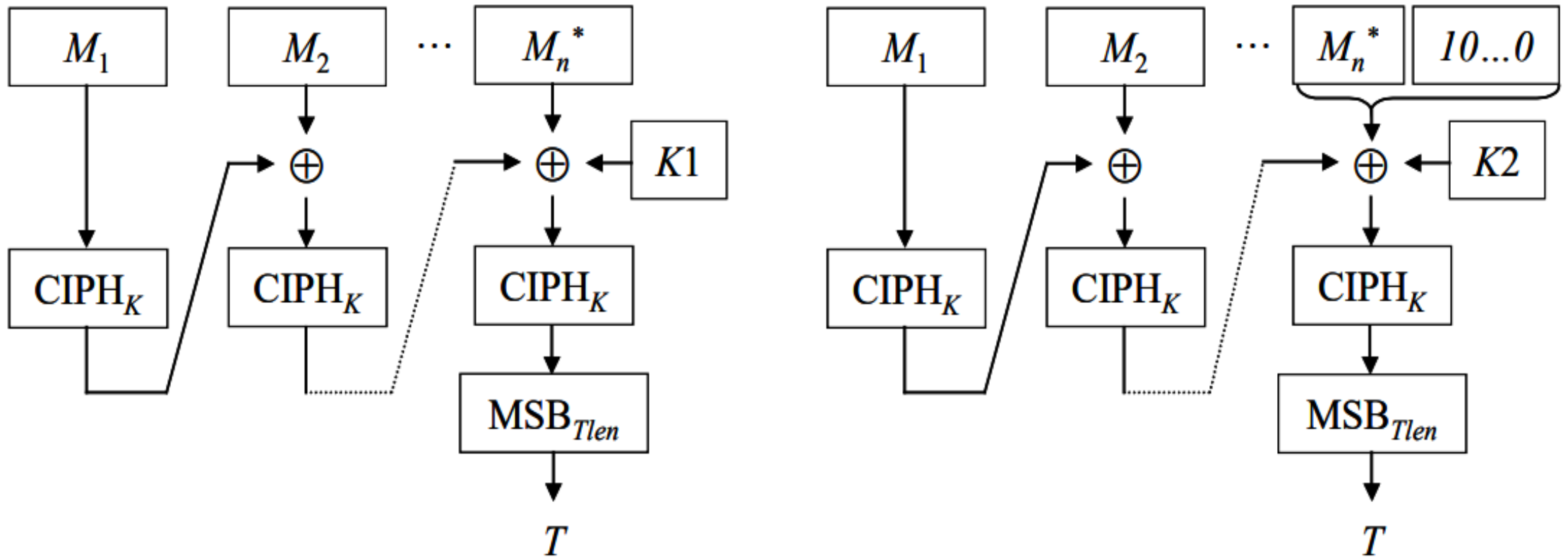
Message Authentication Code

other <u>block ciphers</u>	SEED , RC5, SHACAL-2, SIMECK , SIMON (64/128) , Skipjack, SPECK (64/128) , Simeck , SM4 , Threefish (256/512/1024) , Triple-DES (DES-EDE2 and DES-EDE3) , TEA, XTEA
<u>block cipher modes of operation</u>	ECB, CBC, CBC ciphertext stealing (CTS), CFB, OFB, counter mode (CTR), XTS
<u>message authentication codes</u>	BLAKE2b , BLAKE2s , CMAC , CBC-MAC, DMAC, GMAC (GCM) , HMAC , Poly1305 , SipHash , Two-Track-MAC, VMAC
hash functions	BLAKE2b , BLAKE2s , Keccak (F1600), SHA-1 , SHA-2 , SHA-3, SHAKE (128/256) , SipHash , LSH (128/256) , Tiger , RIPEMD (128/160/256/320), SM3 , WHIRLPOOL
<u>public-key cryptography</u>	RSA , DSA , Deterministic DSA (RFC 6979), ElGamal, Nyberg-Rueppel (NR), Rabin-Williams (RW), EC-based German Digital Signature (ECGDSA), LUC, LUCELG, DLIES (variants of DHAES), ESIGN

CMAC: <https://csrc.nist.gov/pubs/sp/800/38/b/upd1/final>

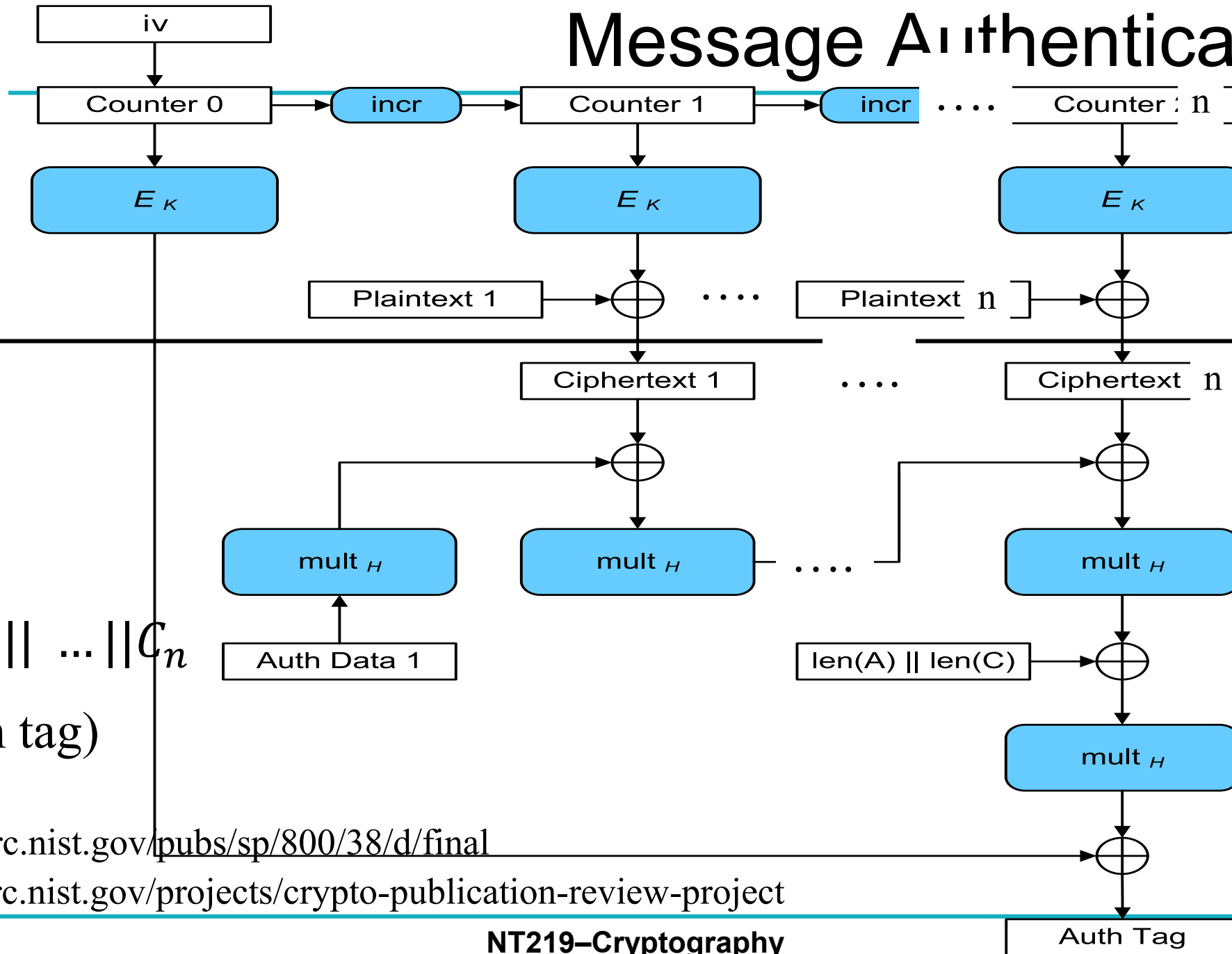
Message Authentication Code

CMAC



Message Authentication Code

GMAC



GCM

$C = C_1 \parallel \dots \parallel C_n$
(C, auth tag)

Page 19 for
Mult_H

<https://csrc.nist.gov/pubs/sp/800/38/d/final>

<https://csrc.nist.gov/projects/crypto-publication-review-project>

Keyed-Hash Message Authentication Code

HMAC

- A hash family (H) use for message authentication
- $MAC(K, M) = H_K(M)$
- The sender and the receiver share secret K
- The sender sends $(M, H_K(M))$
- The receiver receives (X, Y) and verifies that $H_K(X)=Y$, if so, then accepts the message as from the sender
- To be secure, an adversary shouldn't be able to come up with (X', Y') such that $H_K(X')=Y'$.



K

$M, \text{tag} = H(K, M)$



K

$M', \text{tag}' = H(K, M')? = \text{tag}$

Security Requirements for MAC

- Resist the Existential Forgery under Chosen Plaintext Attack
 - Challenger chooses a random key K
 - Adversary chooses a number of messages $M_1, M_2, \dots, M_n$, and obtains $t_j = \text{MAC}(K, M_j)$ for $1 \leq j \leq n$
 - Adversary outputs M' and t'
 - Adversary wins if $\forall j \ M' \neq M_j$, and $t' = \text{MAC}(K, M')$

- Basically, adversary cannot create the MAC for a message for which it hasn't seen an MAC

Constructing MAC from Hash Functions

- Let h be a one-way hash function (SHA2)
- $\text{MAC}(K, M) = h(K \parallel M)$, where $\parallel$ denote concatenation
 - Insecure as MAC
 - Because of the Merkle-Damgard construction for hash functions, given M and $t = h(K \parallel M)$, adversary can compute $M' = M \parallel \text{Pad}(M) \parallel X$ and t' , such that $h(K \parallel M') = t'$

Constructing MAC from Cryptographic Hash Functions

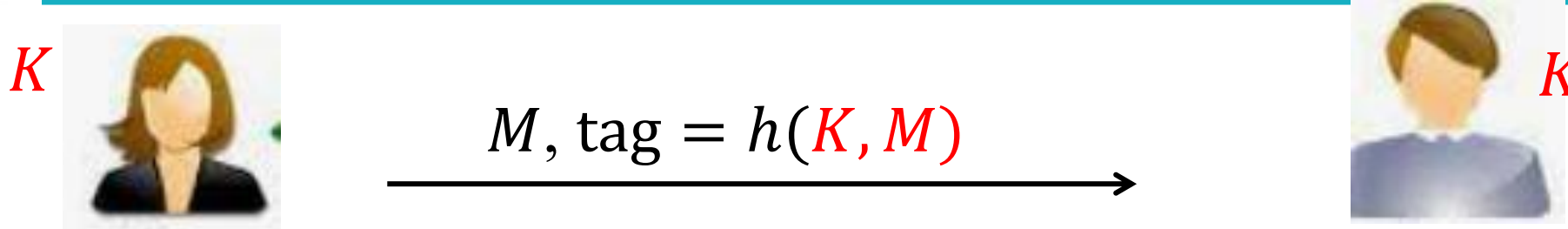
HMAC

$$\text{HMAC}_K[M] = \text{Hash}[(K^+ \oplus \text{opad}) \parallel \text{Hash}[(K^+ \oplus \text{ipad}) \parallel M]]$$

- $K^+ = K \parallel \{0\}^*$ (to B bytes, the input block size of the hash function)
- $\text{ipad} = 0x36 \ 0x36 \ \dots \ 0x36$ (repeated B times)
- $\text{opad} = 0x5C \ 0x5C \ \dots \ 0x5C$ (repeated B times).

At high level, $\text{HMAC}_K[M] = H(K \parallel H(K \parallel M))$

Authentication and Integrity checking



$$h(K, M) = h[(K^+ \oplus \text{opad}) \parallel \text{Hash}[(K^+ \oplus \text{ipad}) \parallel M]]$$

$K^+ = K \parallel \{0\}^*$ (to B bytes, the input **block size** of the hash function)

$\text{ipad} = 0x36 \ 0x36 \ \dots \ 0x36$ (repeated B times)

$\text{opad} = 0x5C \ 0x5C \ \dots \ 0x5C$ (repeated B times).

Authenticate user/end-devices

(1) Password-based Authentication



name, pw

- **Login**



store: name, $h(pw)$

	UserName	HashedSaltedPwd	UserID
1	A-JayBibbins637	0x600FAD66D16A56AF3459D9C996DFF98B	10000
2	A-JayTorain976	0x4E30184141CB6CFF4120D6AC443CCE54	10001
3	AadamDobbin507	0xED2A51BD92E1EE8333314407817CD6F9	10002

- Provide *name, pw'* to server?
 - Locate the row using name
 - Check $h(pw')? = h(pw)$
- ➡ authenticate the user

HMAC Security

- If used with a secure hash functions (e.g., SHA3-256) and according to the specification (key size, and use correct output), no known practical attacks against HMAC

Dictionary attacks on hash function

$h(\text{password}) =$

b1b6a3de29ab907153614683d357b2db943a317d036ff25f702
2d4707109005a

password=?

Guest password	sha256	sha384	sha512	...
abc123	...	...		...
abcd1234	...		...	...
....				

Rainbow table

<https://hashkiller.io/>